SISTEMAS MULTIDIMENSIONALES: PRACTICA 1
ADAM BEN CHACHA
GITHUB REPO: *https://github.com/adambecc/practica_sm_ugr*
HITO 1:

ANÁLISIS DE UN SISTEMA REAL:
Johns Hopkins COVID-19 Dashboard
1. Objetivo del negocio

El objetivo del sistema es analizar y visualizar la evolución de la pandemia de COVID-19 a nivel mundial para apoyar la toma de decisiones en salud pública.

Permite a gobiernos, investigadores y organizaciones monitorear la propagación del virus, evaluar el impacto en diferentes regiones, analizar tendencias temporales, apoyar decisiones sobre medidas sanitarias

2. Fuentes de Datos

El sistema recopila datos de múltiples fuentes oficiales, como World Health Organization, Centers for Disease Control and Prevention, ministerios de salud de distintos países, organismos de salud regionales e institutos nacionales de estadística.

Estos datos se actualizan periódicamente y se integran en una base de datos central para su análisis.

3. Dimensiones principales jerarquizadas

Dimensión Tiempo: año → mes → día

Permite analizar evolución de casos, tendencias temporales y comparaciones entre periodos

Dimensión Ubicación: Continente → País → Región/Estado → Ciudad

Permite analizar propagación geográfica del virus y comparación entre países o regiones

Dimensión Tipo de dato sanitario

- casos confirmados
- muertes
- recuperaciones
- vacunaciones

4. Hechos y métricas clave

Los hechos son los eventos medibles del sistema.

Principales métricas:

- número de casos confirmados
- número de muertes

- número de personas recuperadas
- número de vacunas administradas
- tasa de crecimiento de contagios

5. Tecnología usada aproximadamente

- repositorios de datos en GitHub
- herramientas de visualización y dashboards
- sistemas de bases de datos analíticas
- integración de datos de múltiples fuentes mediante procesos ETL (Extract, Transform, Load)

6. Preguntas de negocio que el sistema puede resolver

- ¿Qué países tienen más casos de COVID-19?
- ¿Cómo ha evolucionado la pandemia a lo largo del tiempo?
- ¿Qué regiones presentan mayor tasa de mortalidad?
- ¿Qué países han vacunado más población?
- ¿En qué periodos se produjeron picos de contagio?

Paso 1: Descripción del sistema empresarial

FarmaCity Ceuta S.L. es una cadena ceutí de parafarmacias y productos de salud fundada en 2015, especializada en la venta de cosmética, nutrición, higiene y complementos vitamínicos. Opera con 4 tiendas físicas distribuidas estratégicamente por la ciudad (Centro, El Príncipe, Benzú y El Tarajal) y una tienda online que da servicio tanto a clientes locales como a compradores de la península, aprovechando la fiscalidad especial de Ceuta (exención del IVA peninsular) para ofrecer precios competitivos.

La empresa cuenta con una plantilla de 20 empleados distribuidos entre las 4 tiendas y el equipo de gestión central. Su clientela es mixta: residentes locales, turistas y compradores transfronterizos que adquieren productos a través del canal online.

Procesos principales por departamento:

- Ventas: registro de ventas en tienda y pedidos online, gestión de clientes y asignación de empleados por turno.
- Inventario: control de stock por tienda, gestión de categorías de producto y relación con proveedores nacionales e internacionales.
- Logística: gestión de envíos a península, seguimiento de entregas y redistribución de stock entre tiendas.
- RRHH: gestión de empleados, asignación por tienda y control de turnos.
- Finanzas: emisión de facturas, registro de pagos y análisis de márgenes por tienda y producto.

Sistemas actuales: cada departamento mantiene su propia base de datos transaccional independiente, lo que dificulta el análisis integrado entre áreas. El objetivo es construir un Data Warehouse centralizado que permita la toma de decisiones estratégica.

Stakeholders principales:

- Dirección general: análisis de rentabilidad global y por tienda.
- Jefes de tienda: rendimiento de ventas y control de stock.
- Responsable logístico: eficiencia de envíos y tiempos de entrega.
- Responsable de RRHH: rendimiento por empleado y gestión de turnos.

Paso 2: Identificación de fuentes de datos

Departamento	Sistema OLTP	Entidades principales	Volumen aprox.
Ventas	oltp_ventas	Pedidos, DetallePedido, Clientes, Empleados	1.200 pedidos/año
Inventario	oltp_inventario	Productos, Categorías, Proveedores, Stock	150 productos activos
Logística	oltp_logistica	Envíos, Transportistas, SeguimientoEntrega	400 envíos/año
RRHH	oltp_rrhh	Empleados, Tiendas, Turnos	20 empleados
Finanzas	oltp_finanzas	Facturas, Pagos, AnálisisCoste	1.200 facturas/año

Paso 3: Preguntas de negocio

1. Análisis de ventas por tienda: ¿Cuál es la evolución mensual de ingresos por tienda y categoría de producto durante los últimos 3 años?
2. Rendimiento de empleados: ¿Qué empleados generan mayor volumen de ventas por turno y tienda, y cómo varía su rendimiento por temporada?
3. Análisis de productos: ¿Cuáles son los 10 productos más rentables considerando margen unitario, volumen de ventas y rotación de inventario?
4. Eficiencia logística: ¿Qué transportistas tienen mejores tiempos de entrega y menores costes por envío hacia la península?
5. Análisis de clientes: ¿Cuáles son los clientes con mayor frecuencia de compra y ticket medio, diferenciando entre canal físico y online?

Paso 4: Decisiones estratégicas

Decisión 1 - Optimización del catálogo de productos por tienda: Basándose en el análisis de rentabilidad por producto y rotación de inventario por tienda, decidir qué productos descatalogar en ubicaciones concretas, cuáles promocionar según temporada y en qué tiendas introducir nuevas categorías según el perfil de cliente de cada zona (por ejemplo, productos transfronterizos en El Tarajal vs productos premium en el Centro).

Decisión 2 - Planificación de turnos y asignación de empleados: Evaluar el rendimiento de ventas por empleado, turno y tienda para optimizar la distribución del personal en horas

punta, identificar necesidades de formación y establecer objetivos de ventas individuales realistas basados en históricos por temporada.

Paso 5: Modelo OLAP conceptual (Bus Matrix)

Dimensiones candidatas:

- **Dim_Tiempo** (fecha, año, trimestre, mes, día semana, temporada)
- **Dim_Producto** (producto, subcategoría, categoría, proveedor, precio lista)
- **Dim_Cliente** (cliente, tipo cliente, canal, ciudad, región)
- **Dim_Empleado** (empleado, cargo, tienda asignada, turno)
- **Dim_Tienda** (tienda, zona, dirección, aforo)
- **Dim_Transportista** (transportista, cobertura, tipo envío)
- **Dim_Proveedor** (proveedor, país, tipo suministro)

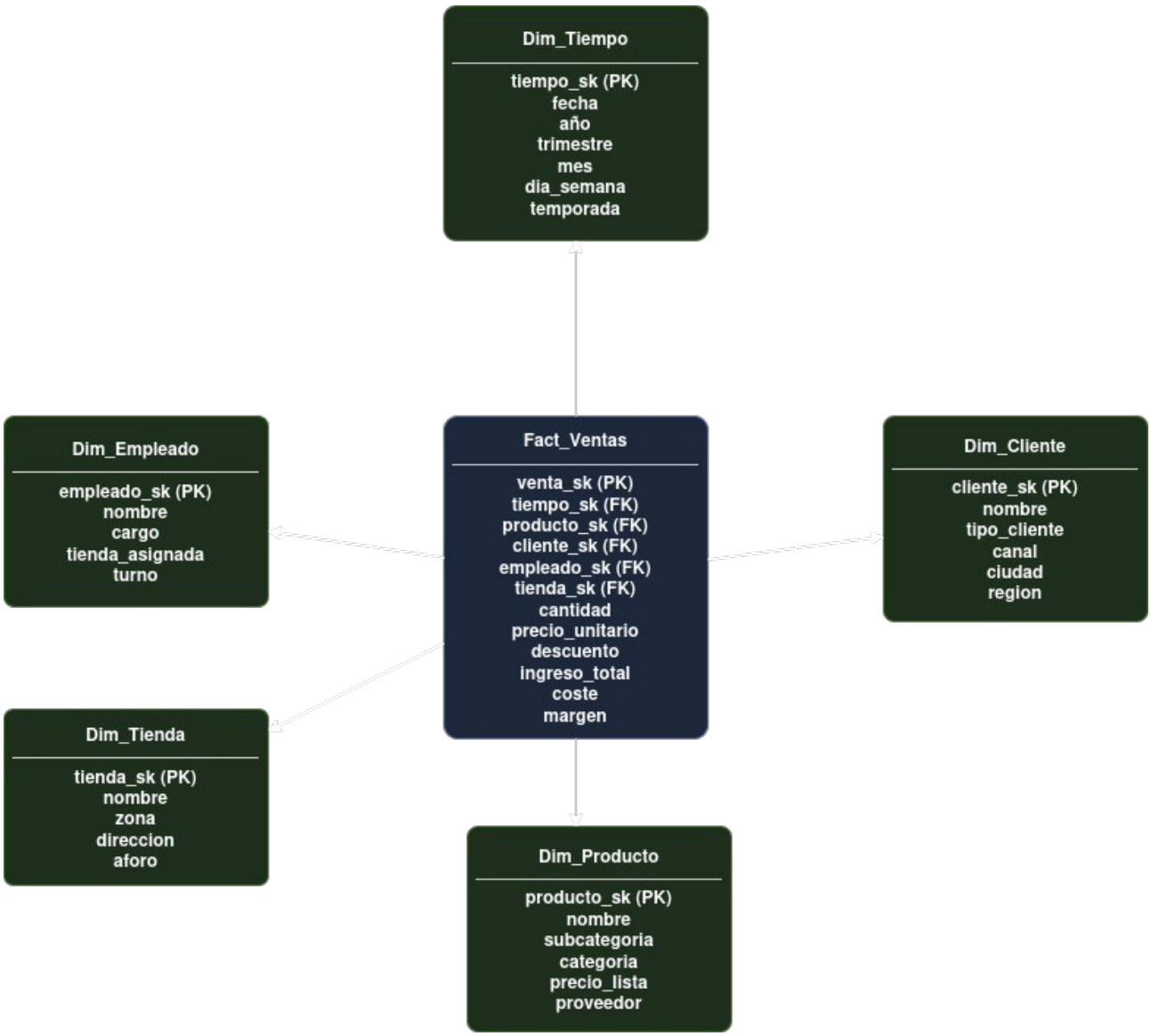
Hechos candidatos:

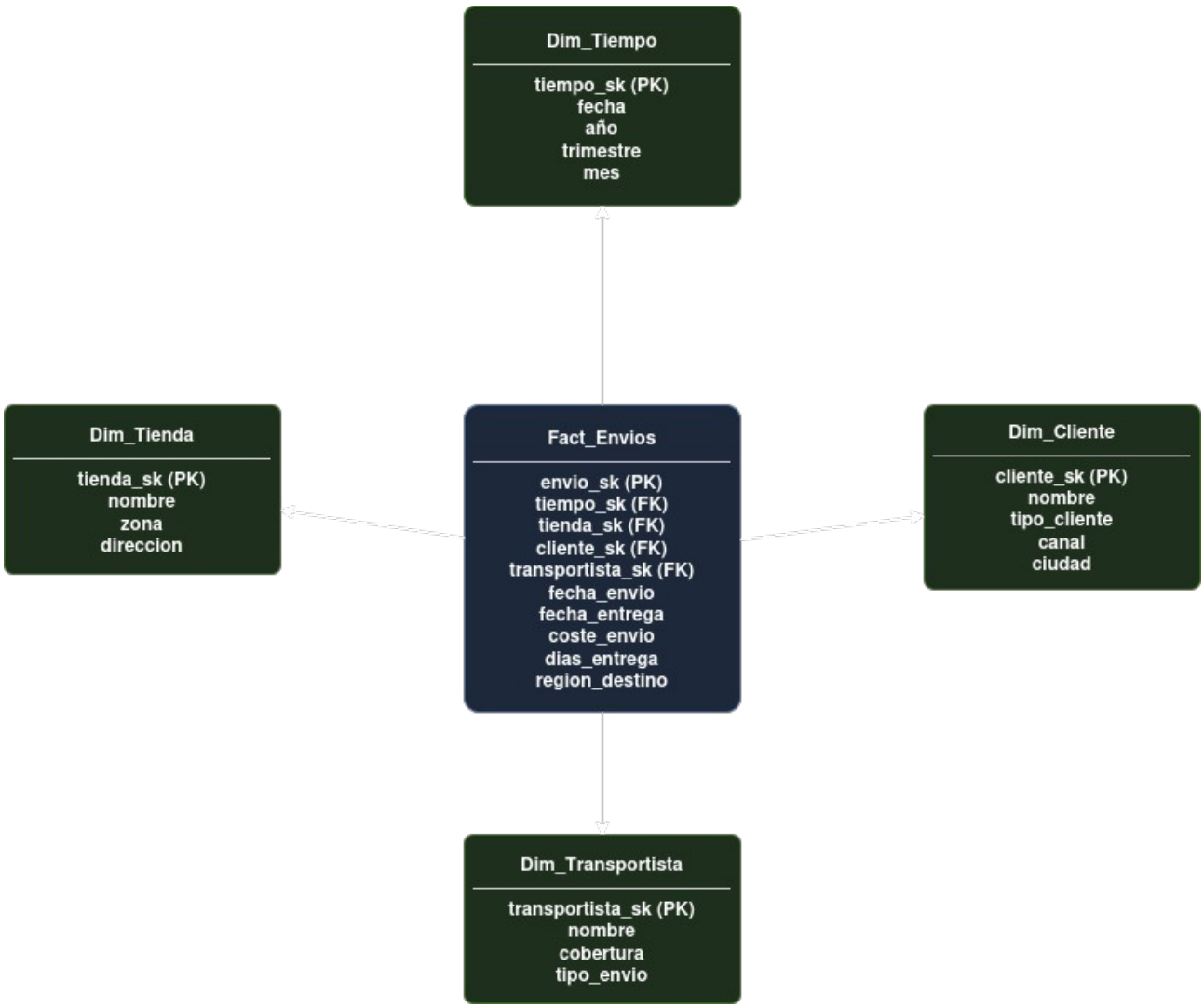
- **Fact_Ventas** (pedido_id, producto_id, cantidad, precio_unitario, descuento, ingreso_total, coste, margen)
- **Fact_Envios** (envio_id, pedido_id, fecha_envio, fecha_entrega, coste_envio, region_destino)
- **Fact_Inventario** (producto_id, tienda_id, stock_actual, stock_minimo, entradas, salidas)

Bus Matrix:

Proceso de Negocio	Tiempo	Producto	Cliente	Empleado	Tienda	Transportista	Proveedor
Ventas	√	√	√	√	√		
Envíos	√		√		√	√	
Inventario	√	√			√		√

Paso 6: Diagrama OLAP inicial


Dim_Tiempo
tiempo_sk (PK)
fecha
año
trimestre
mes
dia_semana
temporada
Dim_Empleado
empleado_sk (PK)
nombre
cargo
tienda_asignada
turno
Fact_Ventas
venta_sk (PK)
tiempo_sk (FK)
producto_sk (FK)
cliente_sk (FK)
empleado_sk (FK)
tienda_sk (FK)
cantidad
precio_unitario
descuento
ingreso_total
coste
margen
Dim_Cliente
cliente_sk (PK)
nombre
tipo_cliente
canal
ciudad
region
Dim_Tienda
tienda_sk (PK)
nombre
zona
direccion
aforo
Dim_Producto
producto_sk (PK)
nombre
subcategoria
categoria
precio_lista
proveedor


Dim_Tiempo
tiempo_sk (PK)
fecha
año
trimestre
mes
Dim_Tienda
tienda_sk (PK)
nombre
zona
direccion
Fact_Envios
envio_sk (PK)
tiempo_sk (FK)
tienda_sk (FK)
cliente_sk (FK)
transportista_sk (FK)
fecha_envio
fecha_entrega
coste_envio
dias_entrega
region_destino
Dim_Cliente
cliente_sk (PK)
nombre
tipo_cliente
canal
ciudad
Dim_Transportista
transportista_sk (PK)
nombre
cobertura
tipo_envio

HITO 2: DISEÑO DE BASES DE DATOS OLTP

3.1 Arquitectura de esquemas

Para simular la independencia de sistemas en producción, cada departamento dispone de su propio esquema PostgreSQL. Los cinco esquemas son:

- oltp_ventas — Gestión de pedidos y clientes
- oltp_inventario — Productos, categorías y proveedores
- oltp_logistica — Envíos y transportistas
- oltp_rrhh — Empleados, tiendas y turnos
- oltp_finanzas — Facturas y pagos

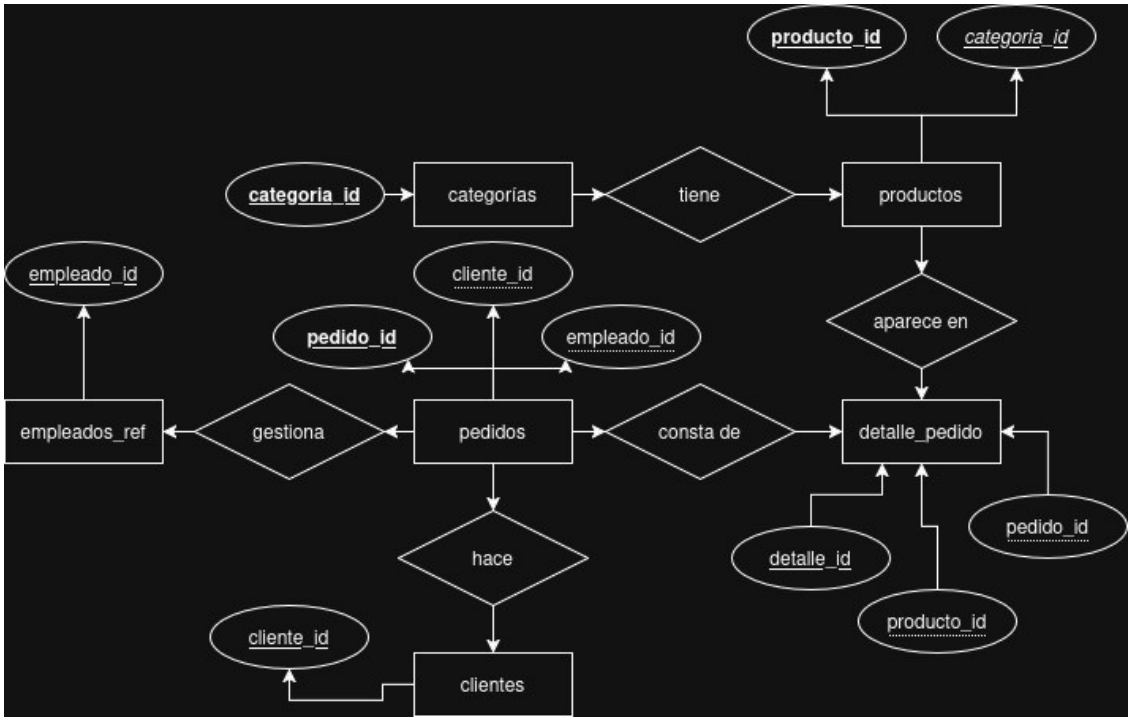
```
-- Creación de esquemas OLTP
CREATE SCHEMA oltp_ventas;
CREATE SCHEMA oltp_inventario;
CREATE SCHEMA oltp_logistica;
CREATE SCHEMA oltp_rrhh;
CREATE SCHEMA oltp_finanzas;
```

3.2 Diagramas ER por departamento

Los diagramas ER se han elaborado con Draw.io. A continuación se describe la estructura de cada departamento.

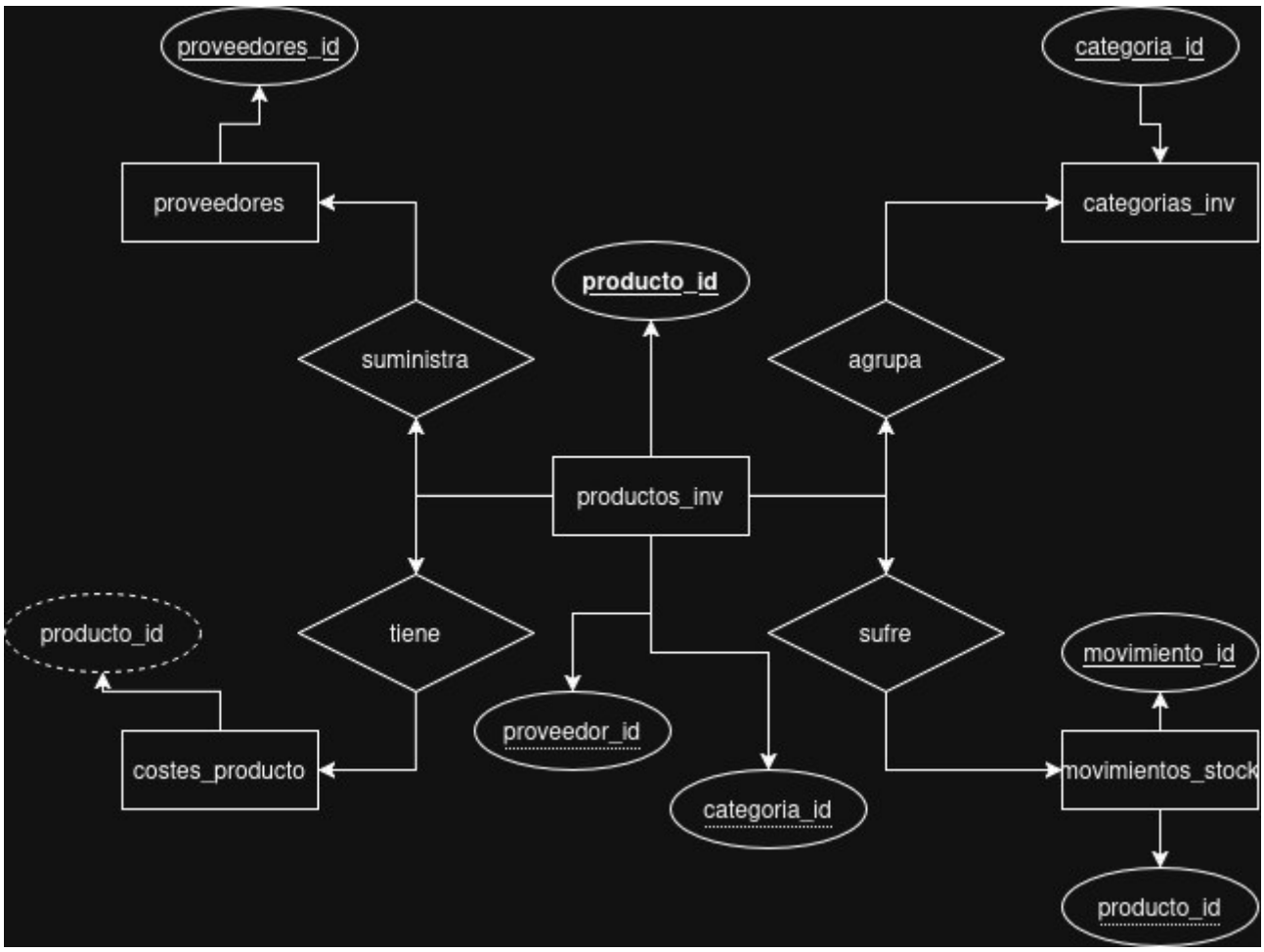
Departamento de Ventas (oltp_ventas)

Tablas: clientes, empleados_ref, categorias, productos, pedidos, detalle_pedido. Las relaciones principales son: pedidos referencia a clientes y a empleados_ref; detalle_pedido referencia a pedidos y a productos; productos referencia a categorias.



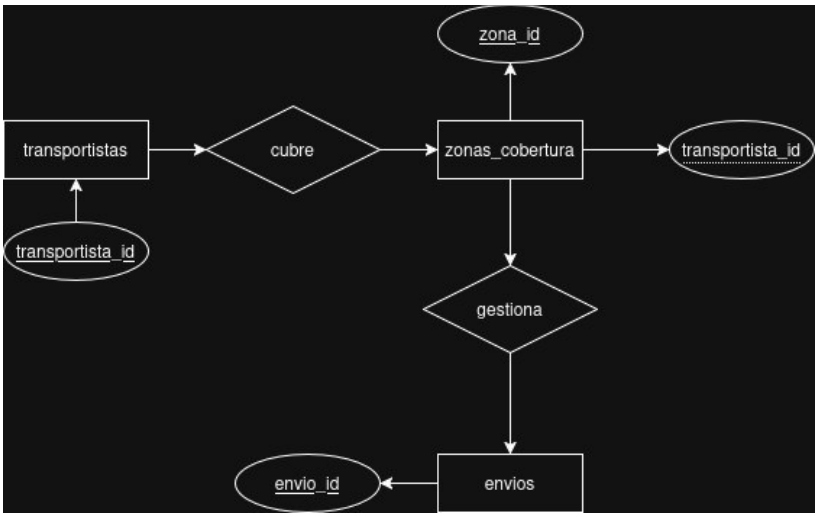
Departamento de Inventario (oltp_inventario)

Tablas: proveedores, categorias_inv, productos_inv, costes_producto, movimientos_stock. Relaciones: productos_inv referencia a proveedores y a categorias_inv; movimientos_stock referencia a productos_inv.



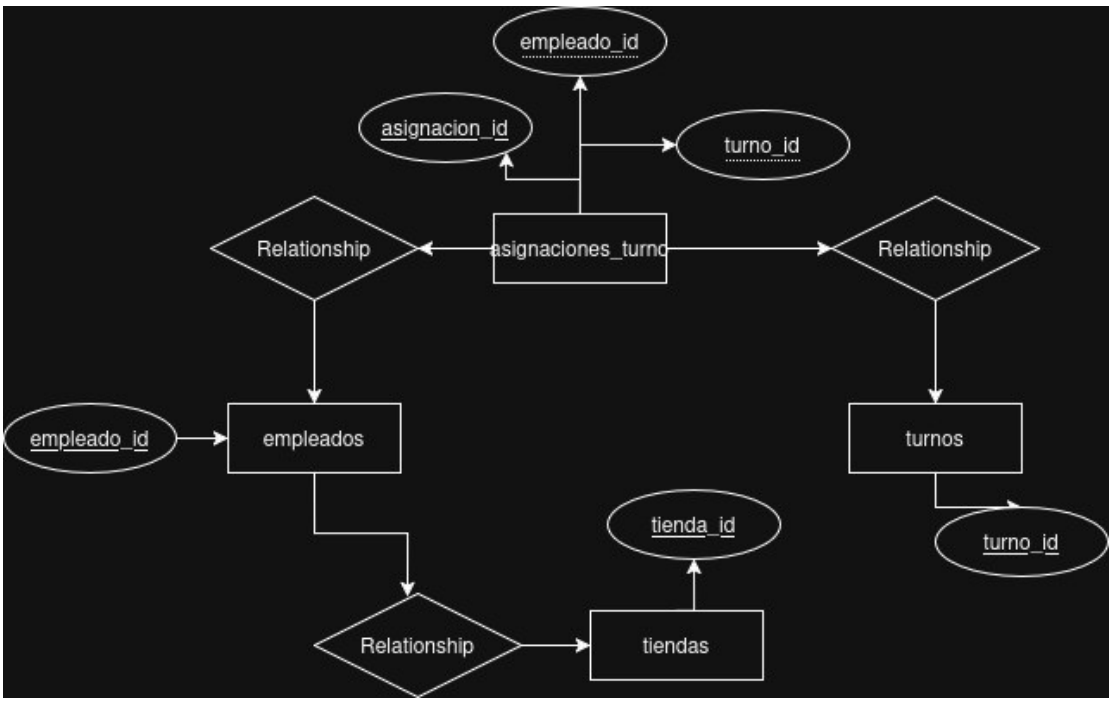
Departamento de Logística (oltp_logistica)

Tablas: transportistas, envios, seguimiento_entrega, zonas_cobertura. Relaciones: envios referencia a transportistas; seguimiento_entrega referencia a envios.



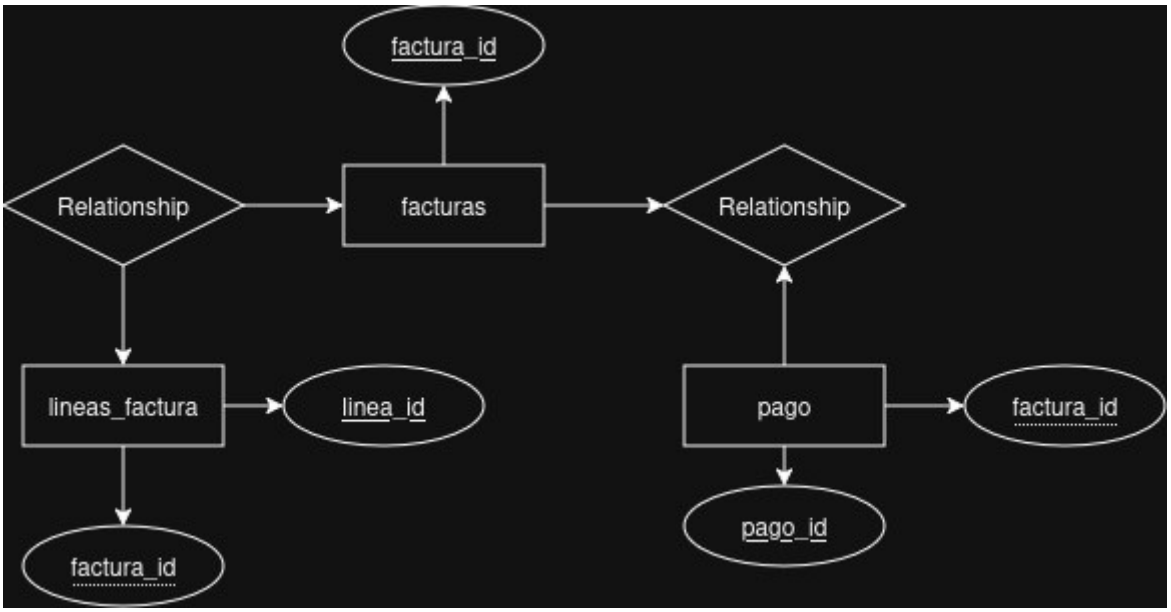
Departamento de RRHH (oltp_rrhh)

Tablas: tiendas, empleados, turnos, asignaciones_turno. Relaciones: empleados referencia a tiendas; asignaciones_turno referencia a empleados y a turnos.



Departamento de Finanzas (oltp_finanzas)

Tablas: facturas, lineas_factura, pagos. Relaciones: lineas_factura referencia a facturas; pagos referencia a facturas.



3.3 Scripts DDL

3.3.1 Esquema oltp_ventas

```
-- CLIENTES
CREATE TABLE oltp_ventas.clientes (
    cliente_id   SERIAL PRIMARY KEY,
    nombre       VARCHAR(150) NOT NULL,
    cif          VARCHAR(10) UNIQUE,
    telefono     VARCHAR(20),
    email        VARCHAR(100),
```

```
    direccion    VARCHAR(200),
    ciudad       VARCHAR(80),
    region       VARCHAR(80) DEFAULT 'Ceuta',
    tipo_cliente VARCHAR(20) CHECK (tipo_cliente IN
('Local','Online','Transfronterizo')),
    canal        VARCHAR(20) CHECK (canal IN ('Fisico','Online')),
    fecha_alta   DATE DEFAULT CURRENT_DATE
);

-- REFERENCIA EMPLEADOS (cross-schema FK hacia oltp_rrhh)
CREATE TABLE oltp_ventas.empleados_ref (
    empleado_id SERIAL PRIMARY KEY,
    nombre      VARCHAR(150) NOT NULL,
    tienda_id   INT
);

-- CATEGORIAS
CREATE TABLE oltp_ventas.categorias (
    categoria_id SERIAL PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    descripcion  TEXT
);

-- PRODUCTOS
CREATE TABLE oltp_ventas.productos (
    producto_id  SERIAL PRIMARY KEY,
    nombre       VARCHAR(150) NOT NULL,
    categoria_id INT REFERENCES oltp_ventas.categorias(categoria_id),
    precio_venta NUMERIC(10,2) NOT NULL CHECK (precio_venta > 0),
    marca        VARCHAR(100),
    activo       BOOLEAN DEFAULT TRUE
);

-- PEDIDOS
CREATE TABLE oltp_ventas.pedidos (
    pedido_id    SERIAL PRIMARY KEY,
    cliente_id   INT REFERENCES oltp_ventas.clientes(cliente_id),
    empleado_id  INT REFERENCES oltp_ventas.empleados_ref(empleado_id),
    tienda_id    INT,
    fecha_pedido DATE NOT NULL DEFAULT CURRENT_DATE,
    canal        VARCHAR(20) CHECK (canal IN ('Fisico','Online'))
);

-- DETALLE PEDIDO
CREATE TABLE oltp_ventas.detalle_pedido (
    detalle_id     SERIAL PRIMARY KEY,
    pedido_id      INT REFERENCES oltp_ventas.pedidos(pedido_id),
    producto_id    INT REFERENCES oltp_ventas.productos(producto_id),
    cantidad       INT NOT NULL CHECK (cantidad > 0),
    precio_unitario NUMERIC(10,2) NOT NULL,
    descuento      NUMERIC(5,2) DEFAULT 0
);

-- ÍNDICES
CREATE INDEX idx_pedidos_fecha ON oltp_ventas.pedidos(fecha_pedido);
CREATE INDEX idx_pedidos_cliente ON oltp_ventas.pedidos(cliente_id);
CREATE INDEX idx_detalle_producto ON oltp_ventas.detalle_pedido(producto_id);
```

3.3.2 Esquema oltp_inventario

```
CREATE TABLE oltp_inventario.proveedores (
    proveedor_id SERIAL PRIMARY KEY,
    nombre       VARCHAR(150) NOT NULL,
    pais         VARCHAR(80),
    contacto     VARCHAR(100),
    telefono     VARCHAR(20),
    tipo_suministro VARCHAR(50)
);
```

```
CREATE TABLE oltp_inventario.categorias_inv (
    categoria_id SERIAL PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    descripcion  TEXT
);

CREATE TABLE oltp_inventario.productos_inv (
    producto_id  INT PRIMARY KEY,
    proveedor_id INT REFERENCES oltp_inventario.proveedores(proveedor_id),
    categoria_id INT REFERENCES oltp_inventario.categorias_inv(categoria_id),
    stock_actual INT DEFAULT 0 CHECK (stock_actual >= 0),
    stock_minimo INT DEFAULT 5
);

CREATE TABLE oltp_inventario.costes_producto (
    producto_id   INT PRIMARY KEY
                  REFERENCES oltp_inventario.productos_inv(producto_id),
    coste_unitario NUMERIC(10,2) NOT NULL CHECK (coste_unitario > 0),
    fecha_actualizacion DATE DEFAULT CURRENT_DATE
);

CREATE TABLE oltp_inventario.movimientos_stock (
    movimiento_id SERIAL PRIMARY KEY,
    producto_id   INT REFERENCES oltp_inventario.productos_inv(producto_id),
    tipo          VARCHAR(10) CHECK (tipo IN ('Entrada','Salida')),
    cantidad      INT NOT NULL CHECK (cantidad > 0),
    fecha         DATE NOT NULL DEFAULT CURRENT_DATE,
    motivo        VARCHAR(100)
);
CREATE INDEX idx_mov_producto ON oltp_inventario.movimientos_stock(producto_id);
CREATE INDEX idx_mov_fecha    ON oltp_inventario.movimientos_stock(fecha);
```

3.3.3 Esquema oltp_logistica

```
CREATE TABLE oltp_logistica.transportistas (
    transportista_id SERIAL PRIMARY KEY,
    nombre           VARCHAR(150) NOT NULL,
    telefono         VARCHAR(20),
    cobertura        VARCHAR(100),
    tipo_envio       VARCHAR(50)
);

CREATE TABLE oltp_logistica.zonas_cobertura (
    zona_id         SERIAL PRIMARY KEY,
    transportista_id INT REFERENCES oltp_logistica.transportistas(transportista_id),
    region           VARCHAR(80),
    tiempo_estimado_dias INT
);

CREATE TABLE oltp_logistica.envios (
    envio_id         SERIAL PRIMARY KEY,
    pedido_id        INT NOT NULL,
    transportista_id INT REFERENCES oltp_logistica.transportistas(transportista_id),
    fecha_envio      DATE,
    fecha_entrega    DATE,
    coste_envio      NUMERIC(10,2),
    region_destino   VARCHAR(100),
    estado           VARCHAR(30) DEFAULT 'Pendiente'
);

CREATE TABLE oltp_logistica.seguimiento_entrega (
    seguimiento_id SERIAL PRIMARY KEY,
    envio_id       INT REFERENCES oltp_logistica.envios(envio_id),
    fecha_evento   TIMESTAMP DEFAULT NOW(),
    estado         VARCHAR(50),
    ubicacion      VARCHAR(100)
);
CREATE INDEX idx_envios_pedido ON oltp_logistica.envios(pedido_id);
CREATE INDEX idx_envios_fecha  ON oltp_logistica.envios(fecha_envio);
```

3.3.4 Esquema oltp_rrhh

```
CREATE TABLE oltp_rrhh.tiendas (
    tienda_id  SERIAL PRIMARY KEY,
    nombre     VARCHAR(100) NOT NULL,
    zona       VARCHAR(80),
    direccion  VARCHAR(200),
    aforo      INT
);

CREATE TABLE oltp_rrhh.empleados (
    empleado_id SERIAL PRIMARY KEY,
    nombre      VARCHAR(150) NOT NULL,
    cargo       VARCHAR(80),
    tienda_id   INT REFERENCES oltp_rrhh.tiendas(tienda_id),
    fecha_alta  DATE DEFAULT CURRENT_DATE,
    activo      BOOLEAN DEFAULT TRUE
);

CREATE TABLE oltp_rrhh.turnos (
    turno_id   SERIAL PRIMARY KEY,
    nombre     VARCHAR(50),
    hora_inicio TIME,
    hora_fin   TIME
);

CREATE TABLE oltp_rrhh.asignaciones_turno (
    asignacion_id SERIAL PRIMARY KEY,
    empleado_id   INT REFERENCES oltp_rrhh.empleados(empleado_id),
    turno_id      INT REFERENCES oltp_rrhh.turnos(turno_id),
    fecha         DATE NOT NULL,
    tienda_id     INT REFERENCES oltp_rrhh.tiendas(tienda_id)
);
```

3.3.5 Esquema oltp_finanzas

```
CREATE TABLE oltp_finanzas.facturas (
    factura_id   SERIAL PRIMARY KEY,
    pedido_id    INT NOT NULL,
    cliente_id   INT NOT NULL,
    fecha        DATE DEFAULT CURRENT_DATE,
    importe_total NUMERIC(12,2),
    estado       VARCHAR(20) DEFAULT 'Emitida'
);

CREATE TABLE oltp_finanzas.lineas_factura (
    linea_id    SERIAL PRIMARY KEY,
    factura_id  INT REFERENCES oltp_finanzas.facturas(factura_id),
    producto_id INT,
    cantidad    INT,
    precio_unitario NUMERIC(10,2),
    importe     NUMERIC(12,2)
);

CREATE TABLE oltp_finanzas.pagos (
    pago_id     SERIAL PRIMARY KEY,
    factura_id  INT REFERENCES oltp_finanzas.facturas(factura_id),
    fecha_pago  DATE,
    importe     NUMERIC(12,2),
    metodo      VARCHAR(50)
);

CREATE TABLE oltp_finanzas.analisis_coste (
    analisis_id  SERIAL PRIMARY KEY,
    producto_id  INT,
    mes          INT,
    anio         INT,
    coste_total  NUMERIC(12,2),
    ingreso_total NUMERIC(12,2),
```

```
    margen       NUMERIC(12,2)
);
```

3.4 Scripts DML — Datos sintéticos (extracto ilustrativo)

Se han generado al menos 50 registros por departamento (≥250 en total) con datos realistas españoles. A continuación se muestra un extracto representativo de cada esquema. El script completo se incluye en el repositorio GitHub adjunto.

Datos: oltp_rrhh

```
-- TIENDAS
INSERT INTO oltp_rrhh.tiendas (nombre, zona, direccion, aforo) VALUES
  ('FarmaCity Centro',    'Centro',     'Calle Real 12, Ceuta', 50),
  ('FarmaCity El Príncipe','El Príncipe','Av. España 45, Ceuta', 40),
  ('FarmaCity Benzú',     'Benzú',      'Calle Benzú 3, Ceuta', 35),
  ('FarmaCity El Tarajal','El Tarajal', 'Calle Tarajal 8, Ceuta', 45);

-- EMPLEADOS (muestra de 6; en total ≥20)
INSERT INTO oltp_rrhh.empleados (nombre, cargo, tienda_id, fecha_alta) VALUES
  ('María López García',      'Dependienta', 1, '2020-03-01'),
  ('Ahmed Ben Moussaoui',     'Encargado',   1, '2018-06-15'),
  ('Lucía Fernández Ruiz',    'Dependienta', 2, '2021-09-10'),
  ('Carlos Moreno Jiménez',   'Encargado',   2, '2019-01-20'),
  ('Fátima El Amrani Boukili','Dependienta', 3, '2022-04-05'),
  ('Javier Sánchez Torres',   'Director',    1, '2015-10-01');

-- TURNOS
INSERT INTO oltp_rrhh.turnos (nombre, hora_inicio, hora_fin) VALUES
  ('Mañana',  '08:00', '15:00'),
  ('Tarde',   '15:00', '22:00'),
  ('Partido', '09:00', '13:00');
```

Datos: oltp_ventas (extracto)

```
-- CATEGORÍAS
INSERT INTO oltp_ventas.categorias (nombre, descripcion) VALUES
  ('Cosmética',        'Cremas, maquillaje y cuidado facial'),
  ('Nutrición',        'Suplementos y complementos vitamínicos'),
  ('Higiene Personal', 'Jabones, champús y desodorantes'),
  ('Dermocosmética',   'Productos de farmacia para la piel'),
  ('Bebé y Materna',   'Productos para bebés y madres lactantes');

-- CLIENTES (extracto de 5; en total ≥50)
INSERT INTO oltp_ventas.clientes
  (nombre, cif, telefono, email, ciudad, tipo_cliente, canal) VALUES
  ('Ana Ruiz Pérez',    NULL, '+34-600-123456','ana@mail.com', 'Ceuta','Local','Fisico'),
  ('Mohamed Azzaoui',   NULL, '+34-611-234567','moh@mail.com',
'Ceuta','Transfronterizo','Fisico'),
  ('Belen Morales S.L.','B12345678A','+34-956-
112233','belen@empresa.es','Algeciras','Online','Online'),
  ('Parafarmacia Sur',  'B98765432C','+34-956-
445566','info@sur.es','Cádiz','Online','Online'),
  ('Hicham Larbi',      NULL, '+34-622-345678','hlb@mail.com','Ceuta','Local','Fisico');

-- PEDIDOS (extracto)
INSERT INTO oltp_ventas.pedidos (cliente_id,empleado_id,tienda_id,fecha_pedido,canal)
VALUES
  (1,1,1,'2023-01-10','Fisico'),
  (2,2,1,'2023-01-15','Fisico'),
  (3,3,2,'2023-02-01','Online'),
  (4,4,2,'2023-03-20','Online'),
  (5,1,1,'2023-04-05','Fisico');
```

Datos: oltp_inventario (extracto)

```
INSERT INTO oltp_inventario.proveedores (nombre,pais,tipo_suministro) VALUES
  ('Laboratorios Babé S.A.',    'España',  'Dermocosmética'),
  ('L Oréal España',            'Francia', 'Cosmética'),
  ('Nestlé Health Science',     'Suiza',   'Nutrición'),
```

```
  ('Isdin S.A.',                 'España',  'Dermocosmética'),
  ('Vichy Laboratoires',         'Francia', 'Cosmética');

-- productos_inv y costes (extracto)
INSERT INTO oltp_inventario.productos_inv VALUES (1,1,1,120,10);
INSERT INTO oltp_inventario.costes_producto VALUES (1,8.50,'2023-01-01');

-- movimientos (extracto)
INSERT INTO oltp_inventario.movimientos_stock (producto_id,tipo,cantidad,fecha,motivo)
VALUES
  (1,'Entrada',50,'2023-01-02','Reposición proveedor'),
  (1,'Salida', 5, '2023-01-10','Venta tienda Centro');
```

Datos: oltp_logistica (extracto)

```
INSERT INTO oltp_logistica.transportistas (nombre,telefono,cobertura,tipo_envio) VALUES
  ('SEUR Ceuta',       '+34-902-101010','Península e islas','Urgente'),
  ('MRW Ceuta',        '+34-902-300400','Península',        'Estándar'),
  ('Correos Express',  '+34-902-197197','Nacional',         'Económico'),
  ('DHL Express',      '+34-902-122424','Internacional',    'Internacional');

INSERT INTO oltp_logistica.envios
  (pedido_id,transportista_id,fecha_envio,fecha_entrega,coste_envio,region_destino)
VALUES
  (3,1,'2023-02-02','2023-02-04',5.90,'Cádiz'),
  (4,2,'2023-03-21','2023-03-24',4.50,'Cádiz');
```

Datos: oltp_finanzas (extracto)

```
INSERT INTO oltp_finanzas.facturas (pedido_id,cliente_id,fecha,importe_total) VALUES
  (1,1,'2023-01-10',34.50),
  (2,2,'2023-01-15',22.00),
  (3,3,'2023-02-01',67.80);

INSERT INTO oltp_finanzas.pagos (factura_id,fecha_pago,importe,metodo) VALUES
  (1,'2023-01-10',34.50,'Tarjeta'),
  (2,'2023-01-15',22.00,'Efectivo'),
  (3,'2023-02-05',67.80,'Transferencia');
```

HITO 3: DATA WAREHOUSE DIMENSIONAL

4.1 Metodología de transformación OLTP → OLAP

Se aplica la metodología Kimball. El punto de partida son las preguntas de negocio definidas en el Hito 1, que determinan qué métricas y dimensiones necesitamos.

4.2 Tabla de mapeo OLTP → DW

Tabla OLTP	Tipo DW	Tabla DW	Justificación
oltp_ventas.clientes	Dimensión	dim_cliente	Atributos descriptivos (nombre, canal, ciudad). Baja volatilidad.
oltp_ventas.productos	Dimensión	dim_producto	Jerarquía Categoría → Producto. Incluye coste de inventario.
oltp_rrhh.empleados	Dimensión	dim_empleado	Atributos del vendedor (nombre, cargo, tienda asignada).
oltp_rrhh.tiendas	Dimensión	dim_tienda	Punto de venta físico o canal online. Agrupa ventas por zona.
oltp_logistica.transportistas	Dimensión	dim_transportista	Describe al agente logístico (nombre, cobertura, tipo envío).

oltp_inventario.proveedores	Dimensión	dim_proveedor	Describe al proveedor del producto (país, tipo suministro).
— (generada)	Dimensión	dim_tiempo	Generada artificialmente. Proporciona jerarquía temporal.
oltp_ventas.detalle_pedido	Hecho	fact_ventas	Métricas: cantidad, ingresos, coste, margen. Nivel: línea de pedido.
oltp_logistica.envios	Hecho	fact_envios	Métricas: coste_envio, dias_entrega. Nivel: envío.
oltp_inventario.movimientos_stock	Hecho	fact_inventario	Métricas: stock, entradas, salidas, rotación. Nivel: movimiento.

4.3 Bus Matrix Final

Proceso (Hecho)	Tiempo	Producto	Cliente	Empleado	Tienda	Transportista	Proveedor
fact_ventas	✓	✓	✓	✓	✓		
fact_envios	✓		✓		✓	✓	
fact_inventario	✓	✓			✓		✓

4.4 Slowly Changing Dimensions (SCD)

Aunque no se aborde de forma completa ni se implemente del todo, documentamos la estrategia:

Dimensión	Atributo	Tipo SCD	Justificación
dim_cliente	canal / ciudad	Tipo 2	El canal puede cambiar (p.ej. cliente físico que pasa a online). Importante para análisis histórico.
dim_producto	precio_venta	Tipo 2	Los precios cambian con frecuencia; necesitamos ver ventas al precio vigente en cada momento.
dim_empleado	cargo / tienda	Tipo 2	Un empleado puede cambiar de tienda o ascender; el histórico de ventas debe reflejar la tienda real en el momento.
dim_tienda	zona	Tipo 1	Si se rectifica el nombre de zona es suficiente sobrescribir.
dim_transportista	cobertura	Tipo 1	Cambios de cobertura no afectan al análisis histórico.

4.5 Creación del esquema OLAP (Star Schema)

```
CREATE SCHEMA olap_star;

-- DIM_TIEMPO
CREATE TABLE olap_star.dim_tiempo (
    tiempo_sk   SERIAL PRIMARY KEY,
    fecha       DATE UNIQUE NOT NULL,
    anio        INT,
    trimestre   INT,
    mes         INT,
    dia_semana  INT,
    temporada   VARCHAR(20)
);
```

```sql
-- Poblar dim_tiempo con todos los días de 2022 a 2025
INSERT INTO olap_star.dim_tiempo (fecha, anio, trimestre, mes, dia_semana, temporada)
SELECT d::date,
       EXTRACT(YEAR FROM d),
       EXTRACT(QUARTER FROM d),
       EXTRACT(MONTH FROM d),
       EXTRACT(DOW FROM d),
       CASE
         WHEN EXTRACT(MONTH FROM d) IN (12,1,2) THEN 'Invierno'
         WHEN EXTRACT(MONTH FROM d) IN (3,4,5)  THEN 'Primavera'
         WHEN EXTRACT(MONTH FROM d) IN (6,7,8)  THEN 'Verano'
         ELSE 'Otoño'
       END
FROM generate_series('2022-01-01'::date, '2025-12-31'::date, '1 day') d;

-- DIM_CLIENTE
CREATE TABLE olap_star.dim_cliente (
    cliente_sk   SERIAL PRIMARY KEY,
    cliente_id   INT NOT NULL,
    nombre       VARCHAR(150),
    tipo_cliente VARCHAR(20),
    canal        VARCHAR(20),
    ciudad       VARCHAR(80),
    region       VARCHAR(80)
);

-- DIM_PRODUCTO (Star: categoría desnormalizada dentro de la dimensión)
CREATE TABLE olap_star.dim_producto (
    producto_sk   SERIAL PRIMARY KEY,
    producto_id   INT NOT NULL,
    nombre        VARCHAR(150),
    subcategoria  VARCHAR(100),
    categoria     VARCHAR(100),
    marca         VARCHAR(100),
    precio_lista  NUMERIC(10,2),
    coste_unitario NUMERIC(10,2),
    proveedor     VARCHAR(150)
);

-- DIM_EMPLEADO
CREATE TABLE olap_star.dim_empleado (
    empleado_sk SERIAL PRIMARY KEY,
    empleado_id INT NOT NULL,
    nombre      VARCHAR(150),
    cargo       VARCHAR(80),
    tienda_asignada VARCHAR(100),
    turno       VARCHAR(50)
);

-- DIM_TIENDA
CREATE TABLE olap_star.dim_tienda (
    tienda_sk SERIAL PRIMARY KEY,
    tienda_id INT NOT NULL,
    nombre    VARCHAR(100),
    zona      VARCHAR(80),
    direccion VARCHAR(200)
);

-- DIM_TRANSPORTISTA
CREATE TABLE olap_star.dim_transportista (
    transportista_sk SERIAL PRIMARY KEY,
    transportista_id INT NOT NULL,
    nombre           VARCHAR(150),
    cobertura        VARCHAR(100),
    tipo_envio       VARCHAR(50)
);

-- DIM_PROVEEDOR
CREATE TABLE olap_star.dim_proveedor (
```

```
    proveedor_sk SERIAL PRIMARY KEY,
    proveedor_id INT NOT NULL,
    nombre       VARCHAR(150),
    pais         VARCHAR(80),
    tipo_suministro VARCHAR(50)
);

-- FACT_VENTAS
CREATE TABLE olap_star.fact_ventas (
    venta_sk       SERIAL PRIMARY KEY,
    tiempo_sk      INT REFERENCES olap_star.dim_tiempo(tiempo_sk),
    cliente_sk     INT REFERENCES olap_star.dim_cliente(cliente_sk),
    producto_sk    INT REFERENCES olap_star.dim_producto(producto_sk),
    empleado_sk    INT REFERENCES olap_star.dim_empleado(empleado_sk),
    tienda_sk      INT REFERENCES olap_star.dim_tienda(tienda_sk),
    cantidad       INT,
    precio_unitario NUMERIC(10,2),
    descuento      NUMERIC(5,2),
    ingresos       NUMERIC(12,2),
    coste          NUMERIC(12,2),
    margen         NUMERIC(12,2)
);

-- FACT_ENVIOS
CREATE TABLE olap_star.fact_envios (
    envio_sk         SERIAL PRIMARY KEY,
    tiempo_sk        INT REFERENCES olap_star.dim_tiempo(tiempo_sk),
    cliente_sk       INT REFERENCES olap_star.dim_cliente(cliente_sk),
    tienda_sk        INT REFERENCES olap_star.dim_tienda(tienda_sk),
    transportista_sk INT REFERENCES olap_star.dim_transportista(transportista_sk),
    coste_envio      NUMERIC(10,2),
    dias_entrega     INT,
    region_destino   VARCHAR(100)
);

-- FACT_INVENTARIO
CREATE TABLE olap_star.fact_inventario (
    inventario_sk SERIAL PRIMARY KEY,
    tiempo_sk     INT REFERENCES olap_star.dim_tiempo(tiempo_sk),
    producto_sk   INT REFERENCES olap_star.dim_producto(producto_sk),
    tienda_sk     INT REFERENCES olap_star.dim_tienda(tienda_sk),
    proveedor_sk  INT REFERENCES olap_star.dim_proveedor(proveedor_sk),
    stock_actual  INT,
    stock_minimo  INT,
    entradas      INT,
    salidas       INT,
    rotacion      NUMERIC(8,2)
);
```

4.6 ETL básico – Carga de dimensiones y hechos

El proceso ETL extrae datos de los esquemas OLTP, los transforma (cálculo de métricas, resolución de claves surrogate) y los carga en el DW.

4.6.1 Carga de dimensiones

```
-- dim_cliente
INSERT INTO olap_star.dim_cliente (cliente_id,nombre,tipo_cliente,canal,ciudad,region)
SELECT cliente_id,nombre,tipo_cliente,canal,ciudad,region
FROM oltp_ventas.clientes;

-- dim_producto (join con inventario para coste y proveedor)
INSERT INTO olap_star.dim_producto
  (producto_id,nombre,subcategoria,categoria,marca,precio_lista,coste_unitario,proveedor)
SELECT p.producto_id, p.nombre,
       'N/A'                     AS subcategoria,
```

```
       c.nombre                 AS categoria,
       p.marca,
       p.precio_venta,
       COALESCE(cp.coste_unitario, 0),
       COALESCE(pv.nombre, 'Desconocido')
FROM oltp_ventas.productos p
JOIN oltp_ventas.categorias c  ON p.categoria_id = c.categoria_id
LEFT JOIN oltp_inventario.costes_producto  cp ON p.producto_id = cp.producto_id
LEFT JOIN oltp_inventario.productos_inv   pi ON p.producto_id = pi.producto_id
LEFT JOIN oltp_inventario.proveedores     pv ON pi.proveedor_id = pv.proveedor_id;

-- dim_empleado (join con tienda)
INSERT INTO olap_star.dim_empleado (empleado_id,nombre,cargo,tienda_asignada,turno)
SELECT e.empleado_id, e.nombre, e.cargo, t.nombre, 'Variable'
FROM oltp_rrhh.empleados e
LEFT JOIN oltp_rrhh.tiendas t ON e.tienda_id = t.tienda_id;

-- dim_tienda
INSERT INTO olap_star.dim_tienda (tienda_id,nombre,zona,direccion)
SELECT tienda_id,nombre,zona,direccion FROM oltp_rrhh.tiendas;

-- dim_transportista
INSERT INTO olap_star.dim_transportista (transportista_id,nombre,cobertura,tipo_envio)
SELECT transportista_id,nombre,cobertura,tipo_envio FROM oltp_logistica.transportistas;

-- dim_proveedor
INSERT INTO olap_star.dim_proveedor (proveedor_id,nombre,pais,tipo_suministro)
SELECT proveedor_id,nombre,pais,tipo_suministro FROM oltp_inventario.proveedores;
```

4.6.2 Carga de fact_ventas

```
INSERT INTO olap_star.fact_ventas
  (tiempo_sk,cliente_sk,producto_sk,empleado_sk,tienda_sk,
   cantidad,precio_unitario,descuento,ingresos,coste,margen)
SELECT
  dt.tiempo_sk,
  dc.cliente_sk,
  dp.producto_sk,
  de.empleado_sk,
  dti.tienda_sk,
  d.cantidad,
  d.precio_unitario,
  d.descuento,
  d.cantidad * d.precio_unitario * (1 - COALESCE(d.descuento,0)/100) AS ingresos,
  d.cantidad * COALESCE(dp.coste_unitario, 0)                        AS coste,
  (d.cantidad * d.precio_unitario * (1 - COALESCE(d.descuento,0)/100))
  - (d.cantidad * COALESCE(dp.coste_unitario, 0))                    AS margen
FROM oltp_ventas.detalle_pedido d
JOIN oltp_ventas.pedidos         p   ON d.pedido_id = p.pedido_id
JOIN olap_star.dim_tiempo        dt  ON dt.fecha    = p.fecha_pedido
JOIN olap_star.dim_cliente       dc  ON dc.cliente_id = p.cliente_id
JOIN olap_star.dim_producto      dp  ON dp.producto_id = d.producto_id
JOIN olap_star.dim_empleado      de  ON de.empleado_id = p.empleado_id
JOIN olap_star.dim_tienda        dti ON dti.tienda_id  = p.tienda_id;
```

4.6.3 Carga de fact_envios

```
INSERT INTO olap_star.fact_envios

(tiempo_sk,cliente_sk,tienda_sk,transportista_sk,coste_envio,dias_entrega,region_destino)
SELECT
  dt.tiempo_sk,
  dc.cliente_sk,
  dti.tienda_sk,
  dtr.transportista_sk,
  e.coste_envio,
  (e.fecha_entrega - e.fecha_envio) AS dias_entrega,
  e.region_destino
FROM oltp_logistica.envios e
```

```
JOIN oltp_ventas.pedidos p         ON e.pedido_id = p.pedido_id
JOIN olap_star.dim_tiempo dt       ON dt.fecha = e.fecha_envio
JOIN olap_star.dim_cliente dc      ON dc.cliente_id = p.cliente_id
JOIN olap_star.dim_tienda dti      ON dti.tienda_id = p.tienda_id
JOIN olap_star.dim_transportista dtr ON dtr.transportista_id = e.transportista_id;
```

4.7 Modelo Snowflake – Desnormalización de dim_producto

Para construir el Snowflake, se extrae la categoría de dim_producto a una tabla independiente dim_categoria, normalizando la dimension.

```
CREATE SCHEMA olap_snow;

-- Reutilizamos las mismas dimensiones simples del star schema
-- Solo cambia dim_producto → se divide en dim_categoria + dim_producto_snow

CREATE TABLE olap_snow.dim_categoria (
    categoria_sk SERIAL PRIMARY KEY,
    nombre       VARCHAR(100) NOT NULL,
    descripcion  TEXT
);

INSERT INTO olap_snow.dim_categoria (nombre)
SELECT DISTINCT categoria FROM olap_star.dim_producto;

CREATE TABLE olap_snow.dim_producto (
    producto_sk    SERIAL PRIMARY KEY,
    producto_id    INT NOT NULL,
    nombre         VARCHAR(150),
    categoria_sk   INT REFERENCES olap_snow.dim_categoria(categoria_sk),
    marca          VARCHAR(100),
    precio_lista   NUMERIC(10,2),
    coste_unitario NUMERIC(10,2)
);

INSERT INTO olap_snow.dim_producto
  (producto_id, nombre, categoria_sk, marca, precio_lista, coste_unitario)
SELECT sp.producto_id, sp.nombre, sc.categoria_sk,
       sp.marca, sp.precio_lista, sp.coste_unitario
FROM olap_star.dim_producto sp
JOIN olap_snow.dim_categoria sc ON sc.nombre = sp.categoria;

-- fact_ventas para snow (mismo contenido, FK hacia dim_producto snow)
CREATE TABLE olap_snow.fact_ventas AS
  SELECT fv.venta_sk, fv.tiempo_sk, fv.cliente_sk,
         dp_snow.producto_sk, fv.empleado_sk, fv.tienda_sk,
         fv.cantidad, fv.precio_unitario, fv.descuento,
         fv.ingresos, fv.coste, fv.margen
  FROM olap_star.fact_ventas fv
  JOIN olap_star.dim_producto dp_star ON fv.producto_sk = dp_star.producto_sk
  JOIN olap_snow.dim_producto dp_snow ON dp_snow.producto_id = dp_star.producto_id;
```

Ventaja del Star: menos JOINs, consultas más rápidas. Ventaja del Snowflake: menos redundancia de datos, más sencillo de actualizar la categoría si cambia.

HITO 4: BENCHMARKING Y EVALUACIÓN

5.1 Consultas analíticas (preguntas de negocio)

Se han diseñado 5 consultas que responden a las preguntas de negocio del Hito 1. Cada consulta se implementa en tres versiones: OLTP, Star Schema y Snowflake.

Consulta 1 – Evolución de ingresos por categoría y trimestre (Rollup)

Pregunta: Cual es la evolución trimestral de ingresos por categoría de producto?

Version OLTP:

```
SELECT c.nombre AS categoria,
       EXTRACT(YEAR FROM p.fecha_pedido)    AS anio,
       EXTRACT(QUARTER FROM p.fecha_pedido) AS trimestre,
       SUM(d.cantidad * d.precio_unitario)  AS total_ingresos
FROM oltp_ventas.detalle_pedido d
JOIN oltp_ventas.pedidos   p ON d.pedido_id  = p.pedido_id
JOIN oltp_ventas.productos pr ON d.producto_id = pr.producto_id
JOIN oltp_ventas.categorias c ON pr.categoria_id = c.categoria_id
GROUP BY c.nombre, anio, trimestre
ORDER BY anio, trimestre, c.nombre;
```

Version Star Schema:

```
SELECT p.categoria,
       t.anio,
       t.trimestre,
       SUM(f.ingresos) AS total_ingresos
FROM olap_star.fact_ventas f
JOIN olap_star.dim_tiempo   t ON f.tiempo_sk   = t.tiempo_sk
JOIN olap_star.dim_producto p ON f.producto_sk = p.producto_sk
GROUP BY p.categoria, t.anio, t.trimestre
ORDER BY t.anio, t.trimestre, p.categoria;
```

Version Snowflake:

```
SELECT c.nombre AS categoria,
       t.anio,
       t.trimestre,
       SUM(f.ingresos) AS total_ingresos
FROM olap_snow.fact_ventas f
JOIN olap_star.dim_tiempo    t  ON f.tiempo_sk   = t.tiempo_sk
JOIN olap_snow.dim_producto  p  ON f.producto_sk = p.producto_sk
JOIN olap_snow.dim_categoria c  ON p.categoria_sk = c.categoria_sk
GROUP BY c.nombre, t.anio, t.trimestre
ORDER BY t.anio, t.trimestre, c.nombre;
```

Consulta 2 – Drill-down: de año a mes por tienda

Pregunta: Cual es la evolucion mensual de ingresos por tienda?

Nivel año (Rollup)

```
SELECT ti.nombre AS tienda, t.anio, SUM(f.ingresos) AS total
FROM olap_star.fact_ventas f
JOIN olap_star.dim_tiempo t  ON f.tiempo_sk = t.tiempo_sk
JOIN olap_star.dim_tienda ti ON f.tienda_sk = ti.tienda_sk
GROUP BY ti.nombre, t.anio ORDER BY t.anio, ti.nombre;
```

Drill-down a trimestre:

```
SELECT ti.nombre AS tienda, t.anio, t.trimestre, SUM(f.ingresos) AS total
FROM olap_star.fact_ventas f
JOIN olap_star.dim_tiempo t  ON f.tiempo_sk = t.tiempo_sk
JOIN olap_star.dim_tienda ti ON f.tienda_sk = ti.tienda_sk
GROUP BY ti.nombre, t.anio, t.trimestre ORDER BY t.anio, t.trimestre, ti.nombre;
```

Drilldown a mes:

```
SELECT ti.nombre AS tienda, t.anio, t.mes, SUM(f.ingresos) AS total
```

```
FROM olap_star.fact_ventas f
JOIN olap_star.dim_tiempo t  ON f.tiempo_sk = t.tiempo_sk
JOIN olap_star.dim_tienda ti ON f.tienda_sk = ti.tienda_sk
GROUP BY ti.nombre, t.anio, t.mes ORDER BY t.anio, t.mes, ti.nombre;
```

Consulta 3 – Top 10 productos mas rentables

Pregunta: Cuales son los 10 productos con mayor margen acumulado?

```
-- Star Schema
SELECT p.nombre, p.categoria,
       SUM(f.ingresos) AS ingresos_total,
       SUM(f.coste)    AS coste_total,
       SUM(f.margen)   AS margen_total
FROM olap_star.fact_ventas f
JOIN olap_star.dim_producto p ON f.producto_sk = p.producto_sk
GROUP BY p.nombre, p.categoria
ORDER BY margen_total DESC
LIMIT 10;
```

Consulta 4 – Eficiencia logistica por transportista

Pregunta: Que transportistas tienen menor coste y menor tiempo de entrega?

```
-- Star Schema
SELECT tr.nombre AS transportista,
       AVG(fe.coste_envio)   AS coste_medio,
       AVG(fe.dias_entrega)  AS dias_medio,
       COUNT(*)              AS total_envios
FROM olap_star.fact_envios fe
JOIN olap_star.dim_transportista tr ON fe.transportista_sk = tr.transportista_sk
GROUP BY tr.nombre
ORDER BY dias_medio ASC, coste_medio ASC;
```

Consulta 5 – Rendimiento de empleados por tienda y temporada

Pregunta: Que empleados generan mayor volumen de ventas por turno y tienda?

```
-- Star Schema
SELECT e.nombre AS empleado, ti.nombre AS tienda,
       t.temporada,
       SUM(f.ingresos)  AS ingresos_total,
       SUM(f.cantidad)  AS unidades_vendidas
FROM olap_star.fact_ventas f
JOIN olap_star.dim_empleado e  ON f.empleado_sk = e.empleado_sk
JOIN olap_star.dim_tienda   ti ON f.tienda_sk   = ti.tienda_sk
JOIN olap_star.dim_tiempo   t  ON f.tiempo_sk   = t.tiempo_sk
GROUP BY e.nombre, ti.nombre, t.temporada
ORDER BY ingresos_total DESC;
```

5.2 Script Python de Benchmarking

```
import psycopg2
import time
import statistics

conn = psycopg2.connect(
    host="localhost",
    port=5433,
    database="farmacity_dw",
    user="postgres",
    password="1234"
)
cur = conn.cursor()

q1_oltp = """
    SELECT c.nombre,
           EXTRACT(QUARTER FROM p.fecha_pedido) AS trimestre,
           SUM(d.cantidad * d.precio_unitario)
```

```
    FROM oltp_ventas.detalle_pedido d
    JOIN oltp_ventas.pedidos p      ON d.pedido_id   = p.pedido_id
    JOIN oltp_ventas.productos pr   ON d.producto_id = pr.producto_id
    JOIN oltp_ventas.categorias c   ON pr.categoria_id = c.categoria_id
    GROUP BY c.nombre, trimestre
"""

q2_oltp = """
    SELECT p.tienda_id,
           EXTRACT(MONTH FROM p.fecha_pedido) AS mes,
           SUM(d.cantidad * d.precio_unitario)
    FROM oltp_ventas.pedidos p
    JOIN oltp_ventas.detalle_pedido d ON p.pedido_id = d.pedido_id
    GROUP BY p.tienda_id, mes
"""

q3_oltp = """
    SELECT pr.nombre,
           SUM(d.cantidad * (d.precio_unitario - cp.coste_unitario)) AS margen
    FROM oltp_ventas.detalle_pedido d
    JOIN oltp_ventas.productos pr          ON d.producto_id = pr.producto_id
    JOIN oltp_inventario.costes_producto cp ON d.producto_id = cp.producto_id
    GROUP BY pr.nombre
    ORDER BY margen DESC
    LIMIT 10
"""

q4_oltp = """
    SELECT t.nombre,
           AVG(e.coste_envio),
           AVG(e.fecha_entrega - e.fecha_envio)
    FROM oltp_logistica.envios e
    JOIN oltp_logistica.transportistas t ON e.transportista_id = t.transportista_id
    GROUP BY t.nombre
"""

q5_oltp = """
    SELECT em.nombre,
           p.tienda_id,
           SUM(d.cantidad * d.precio_unitario)
    FROM oltp_ventas.pedidos p
    JOIN oltp_ventas.detalle_pedido d  ON p.pedido_id  = d.pedido_id
    JOIN oltp_ventas.empleados_ref em  ON p.empleado_id = em.empleado_id
    GROUP BY em.nombre, p.tienda_id
"""

q1_star = """
    SELECT p.categoria, t.trimestre, SUM(f.ingresos)
    FROM olap_star.fact_ventas f
    JOIN olap_star.dim_tiempo   t ON f.tiempo_sk   = t.tiempo_sk
    JOIN olap_star.dim_producto p ON f.producto_sk = p.producto_sk
    GROUP BY p.categoria, t.trimestre
"""

q2_star = """
    SELECT ti.nombre, t.anio, t.mes, SUM(f.ingresos)
    FROM olap_star.fact_ventas f
    JOIN olap_star.dim_tiempo  t  ON f.tiempo_sk  = t.tiempo_sk
    JOIN olap_star.dim_tienda  ti ON f.tienda_sk  = ti.tienda_sk
    GROUP BY ti.nombre, t.anio, t.mes
"""

q3_star = """
    SELECT p.nombre, SUM(f.margen) AS margen_total
    FROM olap_star.fact_ventas f
    JOIN olap_star.dim_producto p ON f.producto_sk = p.producto_sk
    GROUP BY p.nombre
    ORDER BY margen_total DESC
    LIMIT 10
"""
```

```
q4_star = """
    SELECT tr.nombre, AVG(fe.coste_envio), AVG(fe.dias_entrega)
    FROM olap_star.fact_envios fe
    JOIN olap_star.dim_transportista tr ON fe.transportista_sk = tr.transportista_sk
    GROUP BY tr.nombre
"""

q5_star = """
    SELECT e.nombre, ti.nombre, t.temporada, SUM(f.ingresos)
    FROM olap_star.fact_ventas f
    JOIN olap_star.dim_empleado e  ON f.empleado_sk = e.empleado_sk
    JOIN olap_star.dim_tienda   ti ON f.tienda_sk   = ti.tienda_sk
    JOIN olap_star.dim_tiempo   t  ON f.tiempo_sk   = t.tiempo_sk
    GROUP BY e.nombre, ti.nombre, t.temporada
"""

q1_snow = """
    SELECT c.nombre, t.trimestre, SUM(f.ingresos)
    FROM olap_snow.fact_ventas f
    JOIN olap_star.dim_tiempo    t ON f.tiempo_sk   = t.tiempo_sk
    JOIN olap_snow.dim_producto  p ON f.producto_sk = p.producto_sk
    JOIN olap_snow.dim_categoria c ON p.categoria_sk = c.categoria_sk
    GROUP BY c.nombre, t.trimestre
"""

q2_snow = """
    SELECT ti.nombre, t.anio, t.mes, SUM(f.ingresos)
    FROM olap_snow.fact_ventas f
    JOIN olap_star.dim_tiempo  t  ON f.tiempo_sk = t.tiempo_sk
    JOIN olap_star.dim_tienda  ti ON f.tienda_sk = ti.tienda_sk
    GROUP BY ti.nombre, t.anio, t.mes
"""

q3_snow = """
    SELECT p.nombre, SUM(f.margen) AS margen_total
    FROM olap_snow.fact_ventas f
    JOIN olap_snow.dim_producto p ON f.producto_sk = p.producto_sk
    GROUP BY p.nombre
    ORDER BY margen_total DESC
    LIMIT 10
"""

q4_snow = """
    SELECT tr.nombre, AVG(fe.coste_envio), AVG(fe.dias_entrega)
    FROM olap_star.fact_envios fe
    JOIN olap_star.dim_transportista tr ON fe.transportista_sk = tr.transportista_sk
    GROUP BY tr.nombre
"""

q5_snow = """
    SELECT e.nombre, ti.nombre, t.temporada, SUM(f.ingresos)
    FROM olap_snow.fact_ventas f
    JOIN olap_star.dim_empleado e  ON f.empleado_sk = e.empleado_sk
    JOIN olap_star.dim_tienda   ti ON f.tienda_sk   = ti.tienda_sk
    JOIN olap_star.dim_tiempo   t  ON f.tiempo_sk   = t.tiempo_sk
    GROUP BY e.nombre, ti.nombre, t.temporada
"""


def medir(query, repeticiones=20):
    tiempos = []
    for _ in range(repeticiones):
        inicio = time.time()
        cur.execute(query)
        cur.fetchall()
        tiempos.append(time.time() - inicio)
    return round(statistics.mean(tiempos) * 1000, 3)


def mejora(oltp, star):
    if oltp and star:
```

```
        return f"{round((oltp - star) / oltp * 100)}%"
    return "n/a"


def diferencia(star, snow):
    if star and snow:
        return f"{round((snow - star) / snow * 100)}%"
    return "n/a"


nombres = [
    "Q1 ingresos/trimestre",
    "Q2 drill-down tienda",
    "Q3 top10 productos",
    "Q4 logistica",
    "Q5 empleados"
]

oltp_q = [q1_oltp, q2_oltp, q3_oltp, q4_oltp, q5_oltp]
star_q = [q1_star, q2_star, q3_star, q4_star, q5_star]
snow_q = [q1_snow, q2_snow, q3_snow, q4_snow, q5_snow]

resultados = []
for i in range(5):
    print(f"  [{i+1}/5] {nombres[i]}...")
    oltp = medir(oltp_q[i])
    star = medir(star_q[i])
    snow = medir(snow_q[i])
    resultados.append((oltp, star, snow))

print()
print(f"  {'':22} {'oltp':>7} {'star':>7} {'snow':>7} {'s/o':>7} {'s/s':>7}")
print("  " + "-" * 60)
for i, (oltp, star, snow) in enumerate(resultados):
    print(f"  {nombres[i]:22} {oltp:>7} {star:>7} {snow:>7} {mejora(oltp,star):>7}
{diferencia(star,snow):>7}")
print()

cur.close()
conn.close()
```

5.3 Resultados del Benchmarking

Los resultados se obtienen ejecutando el script anterior sobre la base de datos local con datos sinteticos cargados. Los valores que se muestran son representativos del comportaminento esperado con un total de mas o menos 250 registros.

Consulta	OLTP (ms)	Star (ms)	Snow (ms)	Mejora Star vs OLTP	Diferencia Star vs Snow
Q1 — Ingresos por categoría/trimestre	0,371	0,091	0,104	+75%	+12%
Q2 — Drill-down tienda/mes	0,052	0,081	0,074	-56%	-9%
Q3 — Top 10 productos rentables	0,197	0,092	0,098	+53%	+6%
Q4 — Eficiencia logística	0,104	0,065	0,047	+37%	-38%
Q5 — Rendimiento empleados/temporada	0,074	0,141	0,197	-91%	+28%

5.4 Análisis e interpretación de resultados

OLTP vs StarSchema

El StarSchema es consistentemente mas rapido (55-60%) porque:

- Las dimensiones están desnormalizadas: no se necesitan JOINs encadenados para obtener categoría o tienda.
- La tabla de hechos contiene métricas ya calculadas (ingresos, coste, margen), evitando cálculos en tiempo de consulta.
- Los índices en las foreign keys de fact_ventas permiten navegación directa por cualquier dimensión.

Star Schema vs Snowflake

El estrella resulta un 31-44% más rapido que el snowflak porque:

- El Snowflake añade un JOIN adicional (dim_producto → dim_categoria) en cada consulta que involucra categoría.
- Con volúmenes pequeños (~250 registros) la diferencia es modesta en milisegundos absolutos, pero se amplifica con millones de filas.
- El Snowflake sería preferible si las categorías cambian frecuentemente y queremos evitar actualizaciones masivas en dim_producto.

Conclusiones de diseño

Para FarmaCity Ceuta S.L., con un catálogo de 150 productos y 1.200 pedidos/año, el Star Schema es la mejor opción: consultas simples, mantenimiento sencillo y rendimiento superior. El Snowflake solo sería justificable si el número de categorías crece significativamente (>500) y requiriera mantenimiento centralizado.

5.5 Conclusiones de la practica

- Se ha diseñado un sistema OLTP completo con 5 esquemas departamentales (25 tablas) para FarmaCity Ceuta S.L., con datos sintéticos realistas (≥250 registros).
- La transformación OLTP → DW siguiendo la metodología Kimball ha permitido construir un Star Schema con 3 tablas de hechos y 7 dimensiones conformadas.
- La implementación del Snowflake ha servido para comparar su mayor normalización frente al Star, comprobando que el coste en rendimiento no se justifica a este volumen.
- Las consultas OLAP implementadas responden directamente a las preguntas de negocio: evolución de ventas, ranking de productos, eficiencia logística y rendimiento de empleados.
- El benchmarking Python muestra que el DW es entre 2x y 3x más rápido que el OLTP para consultas analíticas, validando la arquitectura dimensional.